

Two-Tower Recommendation / Matching Direction

1. Context

Initial project direction was too broad because it focused on creating multiple frontend demos for different product ideas. After mentor feedback, the direction should be narrowed down through either:

1. Dataset/domain direction
2. Methodology direction

The methodology direction is now the main direction: **Two-Tower Recommender for Candidate Retrieval / Matching**.

Current project goal:

Build and explain a production-inspired two-tower retrieval system, from offline training to online top-K retrieval, with evaluation and a simple demo/API.

2. Current Main Direction

Main methodology

Two-Tower Recommender for Candidate Retrieval / Matching

Core idea:

- One tower encodes the user/query side into an embedding.
- One tower encodes the item/candidate side into an embedding.
- The system compares both embeddings using dot product or cosine similarity.
- Positive pairs are trained to be close.
- Negative pairs are trained to be far apart.
- After training, item embeddings are precomputed offline.
- Online serving only needs to compute the query/user embedding and retrieve top-K items using vector search.

Basic pipeline:

Training data

- positive user-item pairs
- negative sampling / in-batch negatives
- two-tower model
- user/query embedding + item embedding
- similarity score
- retrieval loss / contrastive loss / cross entropy
- trained retrieval model

Serving pipeline:

Offline:

item tower → precompute all item embeddings → build vector index

Online:

user/query → query tower → query embedding → vector search / ANN → top-K candidates

3. Concepts Already Studied

Two-tower concepts

- Embedding: dense vector representation of user/query/item.
- Dot product: fast similarity score between two embeddings.
- Cosine similarity: similarity based on vector direction.
- In-batch negatives: other positive items in the same batch are reused as negatives.
- False negatives: some negatives may actually be relevant but unseen.
- LogQ correction: adjusts logits using item sampling probability to reduce popularity/sampling bias.
- BOW features: represents user history/profile using token/count features.
- Softmax: converts scores into probabilities.
- Cross entropy: penalizes low probability assigned to the correct item.
- Contrastive loss / InfoNCE: pulls positive pairs closer and pushes negatives apart.

Retrieval and serving concepts

- ANN/vector search: approximate nearest neighbor search for fast top-K retrieval.
- ScaNN: Google vector similarity search library.

- HNSW: graph-based ANN method.
- Quantization: compresses vectors to make search faster and cheaper.
- Precomputed item embeddings: item vectors are generated offline.
- Online query embedding: query/user vector is generated at request time.

Production concepts from Uber and Google

Production recommender systems usually follow:

Retrieval → Ranking → Reranking / filtering / business logic

Key understanding:

- Two-tower mainly solves retrieval/candidate generation.
- Ranking is a later stage that uses richer features and more expensive models.
- Candidate generation must be fast and high-recall.
- Ranking can be slower because it only ranks a smaller candidate set.
- ANN/vector index makes large-scale retrieval possible.
- Model and index must be versioned together because query tower and item tower embeddings must live in the same vector space.

4. Why Two-stage Recommendation Matters

At first, candidate generation + ranking sounds obvious, but it is important because production systems cannot score every item with a heavy ranking model.

Example:

Millions of items

- retrieval selects hundreds/thousands of candidates
- ranking scores only this shortlist
- final recommendations are shown

Important trade-off:

- If retrieval misses a good item, ranking cannot recover it.
- Retrieval quality sets the upper bound for ranking quality.
- Ranking feedback is biased by what retrieval exposed.
- Logs are not objective truth; they are generated by previous recommendation policies.

This leads to important production issues:

- exposure bias
- position bias
- popularity bias
- logged feedback bias
- retrieval-ranking interaction
- cold-start item/user problems

- long-tail item coverage
-

5. Dataset Direction — Two Main Options

The project now considers two dataset directions:

1. **RetailRocket Ecommerce Dataset**
2. **FAR-Trans Financial Asset Recommendation Dataset**

Both can support two-tower retrieval, but they represent different business problems and data challenges.

6. Option A — RetailRocket Ecommerce Dataset

Dataset type

RetailRocket is a real-world ecommerce behavior dataset with implicit user-item interactions.

It contains:

- user/visitor events
- item properties
- category tree

- ecommerce behavior such as view, add-to-cart, and transaction

Why it is suitable

RetailRocket is a strong option because ecommerce recommendation has clear production relevance.

It naturally represents common recommender system challenges:

- implicit feedback
- sparse user-item interactions
- long-tail items
- popularity bias
- many items with few or no clicks
- different event strengths: view < add-to-cart < transaction
- item metadata/category features
- candidate retrieval before ranking

Two-tower framing

User tower:

visitor_id

user history

view/addtocart/transaction sequence

category history

optional BOW user history features

Item tower:

item_id

category_id

item properties

optional item metadata features

Positive pair:

visitor interacted with item

Weighted feedback:

view = weak positive

addtocart = stronger positive

transaction = strongest positive

Possible project tasks

- Build popularity baseline.
- Build item co-occurrence baseline.
- Build two-tower ID-only model.
- Add event weighting.
- Add item category/property features.
- Evaluate Recall@K / NDCG@K / HitRate@K.
- Precompute item embeddings.

- Build FAISS/ScaNN-style retrieval.
- Demo: select visitor/session → show top-K recommended products.

Strength

RetailRocket is easier to connect with mainstream recommender systems and Uber/Google-style retrieval architecture.

Risk

It may require careful preprocessing because ecommerce logs are sparse and noisy. Many users may have very short sessions, and many items may have few interactions.

7. Option B — FAR-Trans Financial Asset Recommendation Dataset

Dataset type

FAR-Trans is a financial asset recommendation dataset. It focuses on recommending financial assets to investors.

It includes:

- customer/investor information
- asset information
- asset pricing time series
- retail investor transaction data

Why it is suitable

FAR-Trans is a more distinctive and domain-specific dataset. It can make the project stand out because it is not a typical movie or ecommerce recommender dataset.

It represents challenges such as:

- investor-asset matching
- transaction-based implicit feedback
- time-sensitive asset behavior
- risk/return context
- customer profile and asset profile matching
- domain constraints and suitability considerations

Two-tower framing

Investor tower:

customer profile

past transactions

portfolio behavior

risk preference if available

investment history features

Asset tower:

asset ID

asset type

price/time-series features

risk/return features

sector/category features

Positive pair:

investor bought/held/traded asset

Possible negative pair:

assets not bought by investor

sampled available assets

hard negatives from same asset category

Possible project tasks

- Build popularity/most-traded baseline.
- Build investor-asset two-tower model.
- Add asset metadata/time-series summary features.
- Compare ID-only vs feature-enhanced model.
- Evaluate Recall@K / NDCG@K / HitRate@K.
- Discuss financial-domain constraints and limitations.
- Demo: select investor profile → retrieve top-K candidate assets.

Strength

FAR-Trans is newer and more unique. It may look stronger from a research/domain-hypothesis perspective because financial asset recommendation is less common than movie/ecommerce recommendation.

Risk

It may be harder to explain and evaluate correctly because financial recommendation has more domain constraints. A simple recommender could be misleading if presented as real investment advice. The project should clearly frame it as a learning prototype, not a real financial advisory system.

8. Dataset Comparison

Criteria	RetailRocket Ecommerce	FAR-Trans Financial Asset
Domain familiarity	Easier	Harder
Recommender relevance	Very high	High but specialized
Production similarity	Ecommerce / Uber-like item recommendation	Financial matching / investment recommendation

Data type	Implicit events	Investor transactions + asset info
Main challenge	long-tail, sparse clicks, popularity bias	domain constraints, time-sensitive assets, suitability
Easy to demo	Yes	Medium
Easy to explain to mentor	Yes	Medium-high if framed well
Uniqueness	Medium	High
Risk	preprocessing complexity	financial interpretation risk
Best use	Main implementation if speed matters	Stretch/alternative if wanting stronger domain uniqueness

9. Recommended Dataset Decision

Recommended main dataset:

RetailRocket Ecommerce Dataset

Reason:

- It is closer to common production recommender systems.
- It clearly shows implicit feedback, long-tail items, and popularity bias.
- It is easier to connect with two-tower retrieval, ANN/vector search, and ranking pipeline.
- It is easier to build a visual demo/API.
- It aligns well with mentor comments about ecommerce items: some products are never clicked, while some are clicked a lot.

Recommended secondary dataset:

FAR-Trans as an optional comparison or future extension

Reason:

- It is newer and more domain-specific.
- It can be used to discuss how two-tower matching changes in financial recommendation.
- It helps show that the methodology generalizes beyond ecommerce.

Final dataset strategy:

Main implementation:

RetailRocket ecommerce two-tower retrieval

Optional extension:

FAR-Trans investor-asset matching comparison

10. Updated Project Scope

Project title

Production-inspired Two-Tower Retrieval System for Ecommerce Recommendation

Alternative title:

Two-Tower Candidate Retrieval for Sparse Implicit Feedback Recommendation

Final project statement

This project studies and implements a production-inspired two-tower retrieval system for recommendation and matching. The main implementation uses ecommerce implicit feedback data to learn user and item embeddings, retrieve top-K products, and evaluate retrieval quality. The project focuses on the applied ML workflow: preprocessing interaction logs, building baselines, training a two-tower model, handling negative sampling and popularity bias, evaluating Recall@K/NDCG@K/HitRate@K, precomputing item embeddings, serving recommendations through vector search, and connecting the design to production recommender patterns used by companies such as Uber and Google.

11. Implementation Plan

Phase 1 — Dataset and preprocessing

- Load RetailRocket events.
- Filter invalid/rare users/items.
- Convert events into implicit feedback.
- Create weighted interactions:
 - view = weak signal
 - add-to-cart = medium signal
 - transaction = strong signal
- Create train/validation/test split.
- Build user and item vocabularies.
- Optionally create item category/property features.

Output:

- cleaned interaction table
- user-item pairs
- train/test split
- dataset statistics
- long-tail analysis

Phase 2 — Baselines

Build simple baselines before deep learning:

1. Popularity baseline
2. Recent/popular category baseline
3. Item co-occurrence or matrix factorization baseline if time allows

Metrics:

- Recall@K
- NDCG@K
- HitRate@K
- coverage
- popularity bias analysis

Purpose:

Show that the two-tower model is compared against meaningful simple methods, not trained in isolation.

Phase 3 — Two-tower retrieval model

Start with simple ID-only two-tower:

User tower:

user_id embedding

Item tower:

item_id embedding

Score:

$\text{dot_product}(\text{user_embedding}, \text{item_embedding})$

Loss:

cross entropy / retrieval loss with in-batch negatives

Then add improvements:

- event weighting
- item category features
- BOW user history features
- LogQ correction
- optional false-negative masking

Compare experiments:

Model	Features	Metrics
Popularity baseline	item frequency	Recall@K / NDCG@K
Two-tower ID-only	user_id + item_id	Recall@K / NDCG@K

Two-tower + category	item category	Recall@K / NDCG@K
Two-tower + BOW history	user history features	Recall@K / NDCG@K
Two-tower + LogQ	bias correction	Recall@K / NDCG@K

Phase 4 — Retrieval serving layer

After training:

Offline:

item tower → all item embeddings → vector index

Online:

user/session → user tower → query embedding → vector search → top-K products

Implementation options:

- brute-force search for small prototype
- FAISS if time allows
- ScaNN if compatible with environment

Build simple demo:

- select user/session
 - show historical interactions
 - retrieve top-K recommended products
 - show scores
 - show event/category explanation
 - compare with popularity baseline
-

Phase 5 — Technical report

Report structure:

1. Problem framing
2. Why candidate retrieval matters
3. Why two-tower architecture
4. Dataset choice and data challenges
5. Baselines
6. Two-tower model architecture
7. Negative sampling / in-batch negatives / LogQ
8. Evaluation metrics
9. Vector search serving architecture
10. Production connection to Uber/Google
11. Dataset comparison: RetailRocket vs FAR-Trans

12. Limitations and future work

Future work:

- ranking model after retrieval
- sequence modeling
- near-real-time user behavior
- GenRec / listwise ranking
- financial asset recommendation using FAR-Trans
- better cold-start handling
- exploration/exploitation

Out of Scope

To keep the project focused and feasible within the internship timeline, the following items are explicitly out of scope:

- No need to implement real GenRec / generative recommender models.
- No need for distributed training.
- No need for DeepSpeed, Ray, or large-scale training infrastructure.
- No need to implement a real Lucene Plus / production search platform.
- No need to build a full production real-time recommendation pipeline.
- No need for a full ranking system after retrieval.
- No need for real-time feature streaming or near-real-time UserContext.
- No need for production-grade A/B testing or online experimentation.
- No need for a full MLOps platform.

- No need to provide financial advice if using FAR-Trans; it should only be framed as a learning prototype / methodology comparison.

Two-Tower Recommender Project: Knowledge and Skills Summary

Source / Article / Notebook	Main Topic	Key Concepts Learned	Practical Skills Learned	How It Supports the Project
Kaggle Notebook (RubensZ)	Practical implementation of Two-Tower models.	Retrieval vs. Ranking vs. Filtering stages.	TF Dataset API, model subclassing, ScaNN indexing.	Provides a baseline code implementation for candidate retrieval.
Google ScaNN Article	Efficient Vector Similarity Search.	MIPS (Maximum Inner Product Search), Anisotropic Vector Quantization.	Optimizing speed-accuracy trade-offs for high-recall retrieval.	Crucial for the "Candidate Retrieval" phase at scale.
Google Cloud Blog	Scaling Deep Retrieval on Vertex AI.	Decoupled inference, TFRS (TensorFlow Recommenders) library.	Deploying query/candidate towers and ANN indexes on cloud infrastructure.	Provides the end-to-end production architecture on Vertex AI.

Google Research (YouTube)	2-Stage Recommender Architecture.	Candidate Generation (Recall) vs. Ranking (Precision) separation.	Designing industrial-scale systems for massive user/item corpora.	Foundational theory for the entire Two-Tower system design.
Uber Innovative Apps	Global TTE Model & System Efficiency.	In-batch negatives, LogQ bias correction, UUID layer sharing.	Using Bag-of-Words (BOW) for eater history and cold-start issues.	Advanced modeling techniques to improve retrieval recall.
Uber Evolution Search	Semantic Search Platform Scaling.	Matryoshka Representation Learning (MRL), Semantic vs. Lexical search.	Managing Blue/Green index deployments and storage optimizations.	Strategies for scaling embeddings and index reliability.
Uber Next-Gen Recommendation	Generative Models & Real-Time Features.	Transformer sequence encoders, Listwise scoring, UserContext.	Engineering near-real-time feature pipelines with low lag.	Provides an advanced roadmap beyond static retrieval.

**ACM
Off-policy
Learning**

Selection Bias in
Recommenders.

Logged feedback
bias, stage
interaction
optimization.

Correcting biases in
training data derived
from previous
systems.

Improves model robustness by
addressing data skew.

**Medium
(Vertex AI /
Kubeflow)**

MLOps and
Deployment
Pipelines.

Kubeflow Pipelines,
automated model
deployment.

Building custom
Vertex AI pipelines for
automated training.

Supports the automation and
maintenance of the project.
